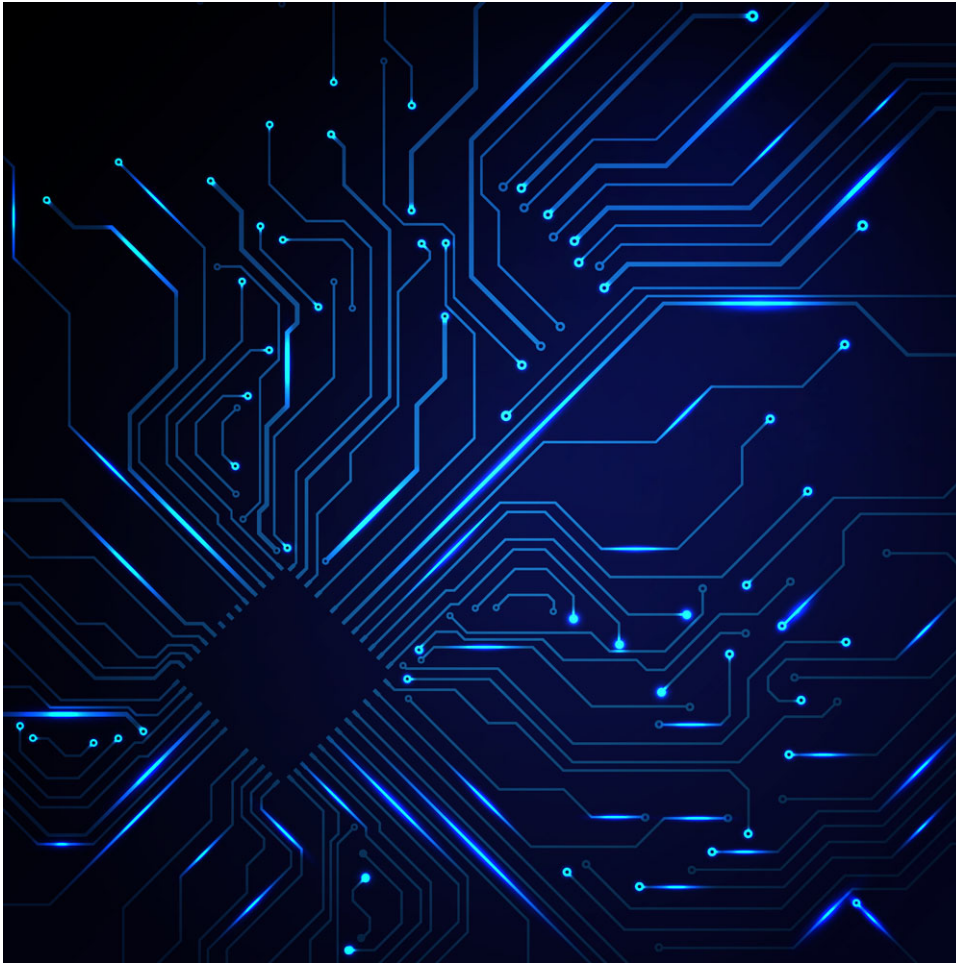




LAB8 COUNTER

Name : 黃翊宏

Student ID : 01357101

Note

1. All files must be converted to PDF format before submission.
2. **Submission deadline:**
Next Tuesday at 1:00 PM
3. Each person submits one assignment.

※ Please use Win + Shift + S for screenshots. Do not take photos of the screen with a phone.



Lab 8-1 :

Main Code Screenshot

counter.sv:

```
V counter.sv X
de0_empty > design > V counter.sv
1  module counter(
2      input clk,
3      input reset,
4      output [2:0] Q,
5      output y
6  );
7
8      logic a, b, c;
9      logic Da, Db, Dc;
10
11     dfflop dff1 (.d(Da), .clk(clk), .reset(reset), .q(a));
12     dfflop dff2 (.d(Db), .clk(clk), .reset(reset), .q(b));
13     dfflop dff3 (.d(Dc), .clk(clk), .reset(reset), .q(c));
14
15     assign Dc = ~c;
16     assign Db = b^c;
17     assign Da = (a & ~b) | (a & ~c) | ((~a & b) & c);
18
19     assign y = a & b & c;
20     assign Q = {a, b, c};
21 endmodule
22
```

dfflop.sv:

```
V dfflop.sv X
de0_empty > design > V dfflop.sv
1  module dfflop(
2      input d,
3      input clk,
4      input reset,
5      output logic q,
6      output logic q_n
7  );
8      //正邊緣觸發 D型正反器
9      always_ff @(posedge clk)
10     begin
11         if (reset) begin
12             q <= 1'b0;
13             q_n <= 1'b1;
14         end
15         else begin
16             q <= d;
17             q_n <= ~d;
18         end
19     end
20
21 endmodule
```

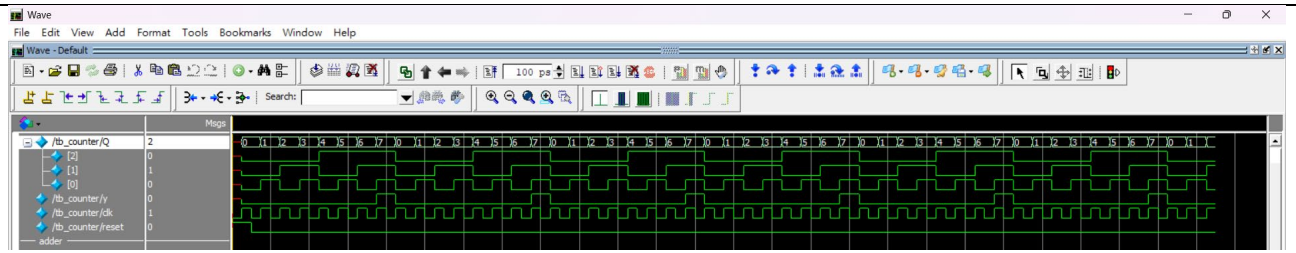
Testbench Screenshot

```
tb_counter.v ×
de0_empty > simulation > tb > tb_counter.v
1  module tb_counter;
2
3      logic [2:0] Q;
4      logic y;
5      logic clk, reset;
6
7      counter u_counter(
8          .clk(clk),
9          .reset(reset),
10         .Q(Q),
11         .y(y)
12     );
13     always #5 clk = ~clk;
14     initial begin
15         clk = 0;
16         reset = 1;
17         #10 reset = 0;
18         #500 $stop;
19     end
20
21 endmodule
```

mcu.sv Code Screenshot

```
mcu.sv ×
de0_empty > design > mcu.sv
36  /*sub_8bit sub1(
40  );*/
41
42  //lab1
43  counter counter1(
44      .clk(BTN[0]),
45      .reset(~BTN[1]),
46      .Q(LED[2:0]),
47      .y(LED[3])
48  );
49
50  //lab2
51  /*logic [3:0] q;
52
53  fourbcounter fourbcounter1(
54      .clk(BTN[0]),
55      .reset(~BTN[1]),
56      .Q(q)
57  );
58
59  ssd ssd1(
60      .a(q),
61      .seg(HEX0)
62  );*/
63
64  /***/
65
66  endmodule
67
```

Waveform and Result Screenshot





Lab 8-2 :

Main Code Screenshot

fourbcounter.sv

```
V fourbcounter.sv X
de0_empty > design > V fourbcounter.sv
1  module fourbcounter(
2      input clk,
3      input reset,
4      output logic [3:0] Q
5  );
6      logic a, b, c, d, Da, Db, Dc, Dd;
7      dfflop dff1(.d(Da), .clk(clk), .reset(reset), .q(a));
8      dfflop dff2(.d(Db), .clk(clk), .reset(reset), .q(b));
9      dfflop dff3(.d(Dc), .clk(clk), .reset(reset), .q(c));
10     dfflop dff4(.d(Dd), .clk(clk), .reset(reset), .q(d));
11
12     assign Dd = (~d);
13     assign Dc = (~c & d) | (c & ~d);
14     assign Db = ((b & ~c) | (b & ~d)) | ((~b & c) & d);
15     assign Da = ((a & ~b) | (a & ~c) | (a & ~d)) | (((~a & b) & c) & d);
16
17     assign Q = {a, b, c, d};
18 endmodule
```

dfflop.sv same as the Lab1.

Testbench Screenshot

```
V fourbcounter.sv  V tb_fourbcounter.sv X
de0_empty > simulation > tb > V tb_fourbcounter.sv
1  module tb_fourbcounter;
2
3      logic [3:0] Q;
4      logic [6:0] seg;
5      logic clk, reset;
6
7      fourbcounter u_fourbcounter(
8          .clk(clk),
9          .reset(reset),
10         .Q(Q)
11     );
12
13     ssd u_ssd(
14         .a(Q),
15         .seg(seg)
16     );
17
18     always #5 clk = ~clk;
19
20     initial begin
21         clk = 0;
22         reset = 1;
23         #10 reset = 0;
24         #500 $stop;
25     end
26
27 endmodule
```

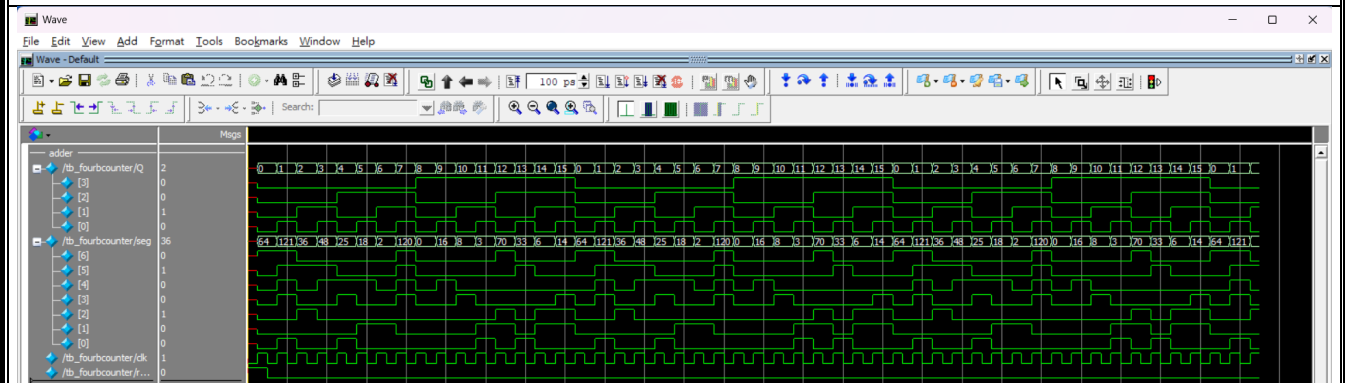
mcu.sv Code Screenshot

```

42 //lab1
43 /*counter counter1(
44     .clk(BTN[0]),
45     .reset(~BTN[1]),
46     .Q(LED[2:0]),
47     .y(LED[3])
48 );*/
49
50 //lab2
51 logic [3:0] q;
52
53 fourbcounter fourbco
54     .clk(BTN[0]),
55     .reset(~BTN[1]),
56     .Q(q)
57 );
58
59 ssd ssd1(
60     .a(q),
61     .seg(HEX0)
62 );
63
64 //*****
65
66 endmodule

```

Waveform and Result Screenshot





Lab Summary and Reflection :

Summary:

In this experiment, we utilized Quartus and the Altera DE0 development board to implement synchronous 3-bit and 4-bit counters in SystemVerilog. For the 3-bit counter, we derived the next-state truth table from the state diagram, coded the sequential logic using `always_ff` constructs, and conducted functional simulation with a dedicated test bench. The outputs `Q[2:0]` and the overflow flag `y` were then mapped to onboard LEDs for live verification, and waveform analysis confirmed correct timing and behavior before hardware synthesis. The 4-bit counter followed the same methodology: after describing its state transitions in SystemVerilog and verifying the counting sequence through simulation, we routed the 4-bit binary output through a seven-segment decoder to the SSD. The resulting display on HEX0 validated the design's correct operation on the DE0 board.

Reflection:

This lab significantly deepened my understanding of sequential logic design and the practical aspects of FPGA implementation. Writing structured `always_ff` blocks and comprehensive testbenches reinforced best practices in hardware description and simulation-driven verification. Deploying the design to the DE0 board introduced real-world considerations such as precise pin assignments, timing constraints, and external decoder integration, highlighting the importance of hardware-aware design decisions. Reviewing and documenting the screenshots, code, and waveforms in my lab report also underscored the value of thorough documentation and visual evidence when troubleshooting complex digital systems.